



Ghulam Ishaq Khan Institute of Engineering Sciences and Technology

Department of Computer Engineering

StreamNest Cloud Migration: Design Proposal for GCP Infrastructure

Prepared for:

Chief Technology Officer, StreamNest

Prepared by:

Ahmed Musharaf

Reg No: 2022067

Course:

CE 408 — Cloud Computing
Assignment 2

Submitted to:

Miss Safia Baloch

Date: May 5, 2026

Cloud infrastructure implemented on Google Cloud Platform (GCP)
using GKE Autopilot, Cloud Storage, and BigQuery.

Contents

1	Executive Summary	4
2	Problem Restatement	4
2.1	Engineering Pain: Bare VMs and Manual Deployments	4
2.2	Data Pain: No Unified Analytics Platform	5
2.3	Business Consequence	5
3	Proposed Architecture	5
3.1	Layer 1: Microservices on GKE Autopilot	6
3.2	Layer 2: Data Lakehouse on GCS + BigQuery	6
4	Microservices on GKE: Implementation	6
4.1	Why Containers Solve the Deployment Problem	6
4.2	Why GKE Autopilot Specifically	7
4.3	Project Scaffold and Configuration	7
4.4	Proof-of-Concept: Content Catalog API	7
4.5	Building and Deploying the Image	8
4.6	Public API Access	9
4.7	Horizontal Scaling	9
4.8	Zero-Downtime Rolling Redeploy	12
5	Data Lakehouse on GCS + BigQuery: Implementation	14
5.1	Why a Lakehouse Architecture	14
5.2	Zone Model	14
5.3	Step 1: Raw Dataset in Cloud Storage	14
5.4	Step 2: BigQuery External Table	15
5.5	Step 3: Curated Aggregated Table	17
6	Scalability & Cost Analysis	19
6.1	GKE Autopilot Scaling	19
6.2	BigQuery Scaling	19
6.3	Cloud Storage Cost Optimisation	20
6.4	Estimated Monthly Cost	20
7	Security & Operations	20
7.1	Identity and Access Management	20
7.2	Network Security	21
7.3	Data Security	21
7.4	Observability	21
8	Roadmap to Production	21
9	Conclusion	22

A	Commands & Manifests Reference	23
A.1	Kubernetes Deployment Manifest	23
A.2	Kubernetes Service Manifest	23
A.3	BigQuery DDL — External Table	24
A.4	BigQuery DDL — Curated Aggregated Table	24
B	Overleaf Upload Checklist	25
C	Resource Cleanup	26

1. Executive Summary

StreamNest’s current platform — microservices running on bare virtual machines combined with unstructured MySQL databases and flat-file data stores — is the root cause of two critical operational failures: slow, error-prone deployments and the data science team’s complete inability to derive timely insights. With the user base scaling from 50,000 to over 2 million active users across Southeast Asia, the existing infrastructure is no longer viable.

This proposal outlines a two-pillar migration to **Google Cloud Platform (GCP)** designed to resolve both problems simultaneously:

1. **Containerized Microservices on GKE Autopilot** — packaging backend services into Docker containers and deploying them on Google Kubernetes Engine, eliminating environment drift and reducing deployment time from hours to minutes with zero downtime.
2. **Data Lakehouse on Cloud Storage + BigQuery** — establishing a unified, scalable data platform where raw event logs land in Cloud Storage and are queryable through BigQuery, enabling the data science team to build recommendation models and generate business reports from clean, structured data.

A working proof-of-concept has been delivered on GCP: a **Content Catalog API** running on GKE that scales horizontally, redeploys with zero downtime, and is accessible via a public endpoint — alongside a queryable BigQuery layer over a 50,000-event simulated dataset that surfaces real business insights. Figure 1 shows the active GCP project used for this proof-of-concept.

2. Problem Restatement

2.1. Engineering Pain: Bare VMs and Manual Deployments

StreamNest’s engineering team currently runs three independent microservices — a user authentication service, a content recommendation engine, and a billing service — each on bare virtual machines. This approach has three compounding problems:

- **Environment inconsistency:** Software libraries and configurations differ between development and production environments, causing bugs that only surface at deployment time.
- **Slow, manual deployments:** Each deployment requires SSH access, manual file transfers, and restarting processes — a process taking hours and prone to human error.
- **No horizontal scaling:** Bare VMs cannot automatically add capacity during peak streaming hours without manual intervention and significant lead time.

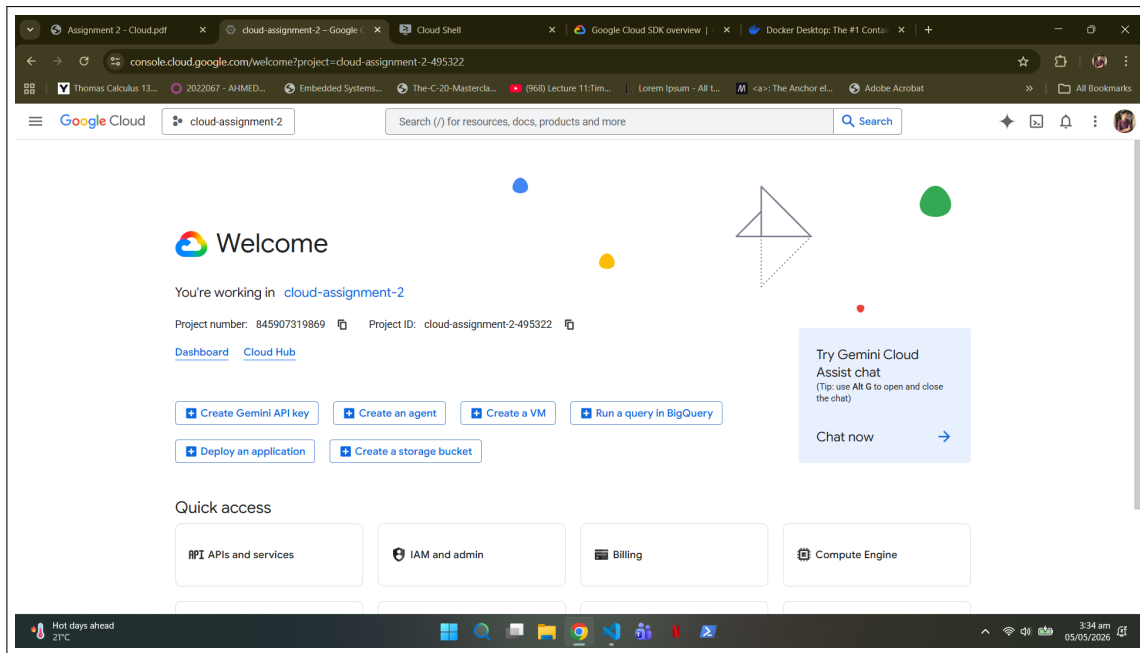


Figure 1: GCP Console project dashboard for `cloud-assignment-2-495322` — the active project used throughout this proof-of-concept.

2.2. Data Pain: No Unified Analytics Platform

The data science team has no shared, structured access to StreamNest’s data:

- Raw event logs (watch history, clickstream, billing records) arrive in different formats — some in real-time bursts during peak hours, others as nightly batch dumps.
- Data is siloed across on-premises MySQL databases and local flat files with no common schema or query interface.
- Building recommendation models or generating executive reports requires weeks of manual data preparation by engineers.

2.3. Business Consequence

At 2 million active users and growing, these problems translate directly to lost revenue: slower feature delivery, degraded video quality during peaks, and recommendations that are stale or absent — all of which drive user churn.

3. Proposed Architecture

The proposed architecture has two independent but complementary layers, both hosted on GCP. Figure 2 shows the complete end-to-end design.

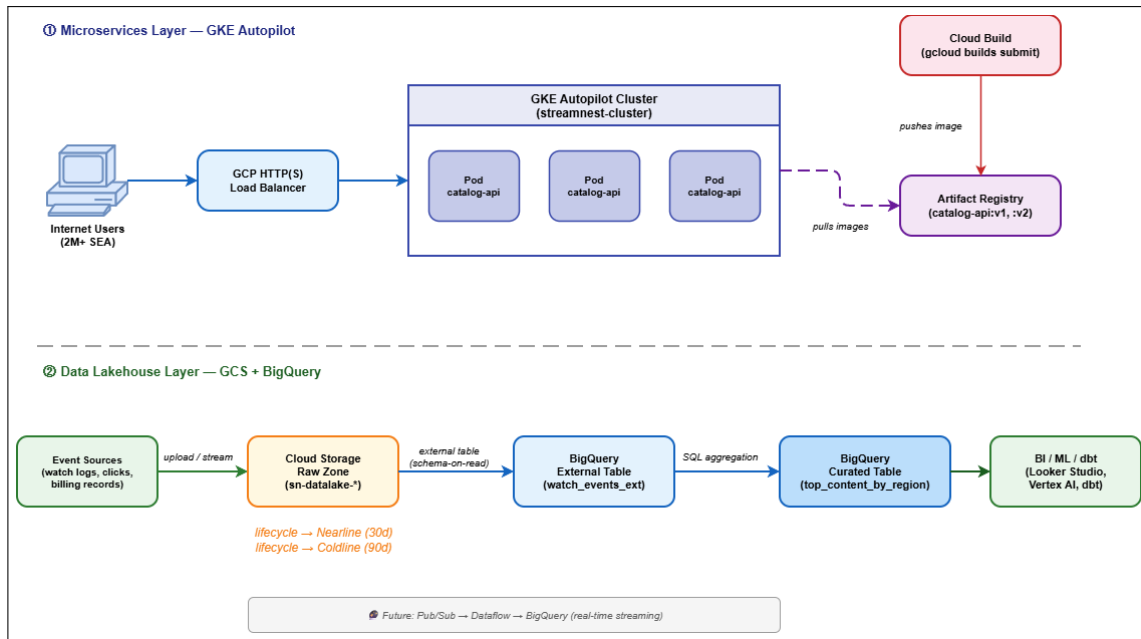


Figure 2: StreamNest GCP Architecture — Microservices Layer (top) handling live user traffic via GKE, and Data Lakehouse Layer (bottom) processing event data through Cloud Storage and BigQuery.

3.1. Layer 1: Microservices on GKE Autopilot

Internet traffic from StreamNest’s 2M+ users reaches a **GCP HTTP(S) Load Balancer**, which distributes requests across container pods inside a **GKE Autopilot cluster**. Container images are built by **Cloud Build** and stored in **Artifact Registry**. The GKE control plane manages pod scheduling, health checks, and rolling updates automatically — no node-level operations are required from the engineering team.

3.2. Layer 2: Data Lakehouse on GCS + BigQuery

Event data from all StreamNest sources lands in **Cloud Storage** as immutable raw files (the *raw zone*). **BigQuery** creates an *external table* directly over this data, enabling SQL queries without moving or copying files. Aggregated results are materialized into **BigQuery native tables** (the *curated zone*), which feed downstream consumers including Looker Studio dashboards, Vertex AI models, and dbt transformation pipelines.

4. Microservices on GKE: Implementation

4.1. Why Containers Solve the Deployment Problem

A Docker container packages an application together with all its dependencies — runtime, libraries, and configuration — into a single immutable artifact. When this artifact is deployed, it behaves identically in development, staging, and production. This directly eliminates the environment drift that currently causes StreamNest’s deployment failures.

Deployment time drops from hours to under two minutes: a new container image is built by **Cloud Build**, pushed to **Artifact Registry**, and a single `kubectl set image` command triggers a rolling update across the cluster.

4.2. Why GKE Autopilot Specifically

Three alternatives were considered:

Option	Benefit	Drawback
Standard GKE	Full node control	Team must manage, patch, and size nodes
GKE Autopilot	Fully managed nodes, pay-per-pod	Slightly less node-level flexibility
Cloud Run	Serverless, zero config	No persistent connections, limited Kubernetes API

Table 1: Comparison of GCP compute options for StreamNest microservices.

GKE Autopilot was chosen because it eliminates all node management overhead while still providing the full Kubernetes API — essential as StreamNest adds more microservices over time. Billing is per pod resource request rather than per VM, which is more cost-efficient for variable streaming workloads.

4.3. Project Scaffold and Configuration

All microservice files were created directly inside Google Cloud Shell, requiring no local installations. Figure 3 shows the complete project directory structure as scaffolded in Cloud Shell.

4.4. Proof-of-Concept: Content Catalog API

A **Content Catalog API** was built in Python/Flask as the representative StreamNest microservice. It exposes three JSON endpoints:

```
GET /          -> service info + pod hostname (load balancing
    evidence)
GET /catalog    -> full JSON content catalog
GET /catalog/<id> -> individual content item lookup
```

The service was containerised with the following Dockerfile:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
```

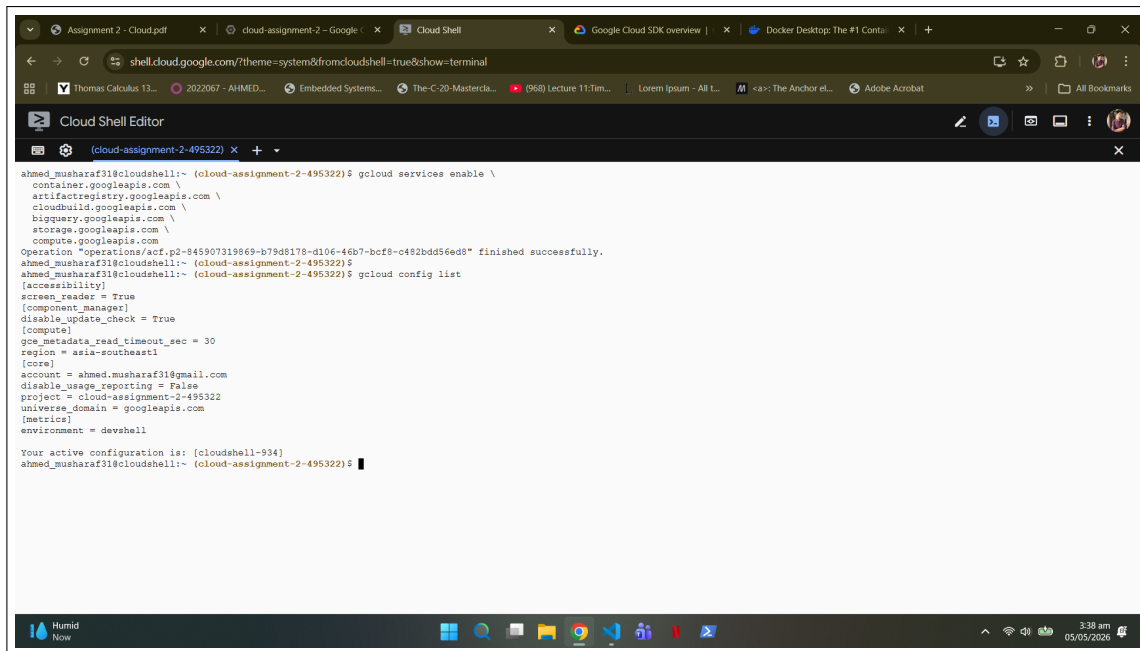



Figure 3: GCP APIs enabled via `gcloud services enable` and project configuration confirmed with `gcloud config list` in Cloud Shell — Phase 0 setup complete.

```

RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
ENV APP_VERSION=v1
EXPOSE 8080
CMD ["gunicorn", "-b", "0.0.0.0:8080", "-w", "2", "app:app"]

```

4.5. Building and Deploying the Image

The image was built entirely on Google's infrastructure using **Cloud Build** — no local Docker installation was required:

```

gcloud builds submit \
  --tag asia-southeast1-docker.pkg.dev/$PROJECT_ID/
  streamnest-repo/catalog-api:v1 .

```

The GKE Autopilot cluster was provisioned in `asia-southeast1` (Singapore) to minimise latency for Southeast Asian users:

```

gcloud container clusters create-auto streamnest-cluster \
  --region=asia-southeast1

```

The Kubernetes deployment was applied using a rolling update strategy that guarantees zero downtime:

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0 # never remove a pod before its
    replacement is ready
```

Figure 4 shows the successful build and deployment output in Cloud Shell.

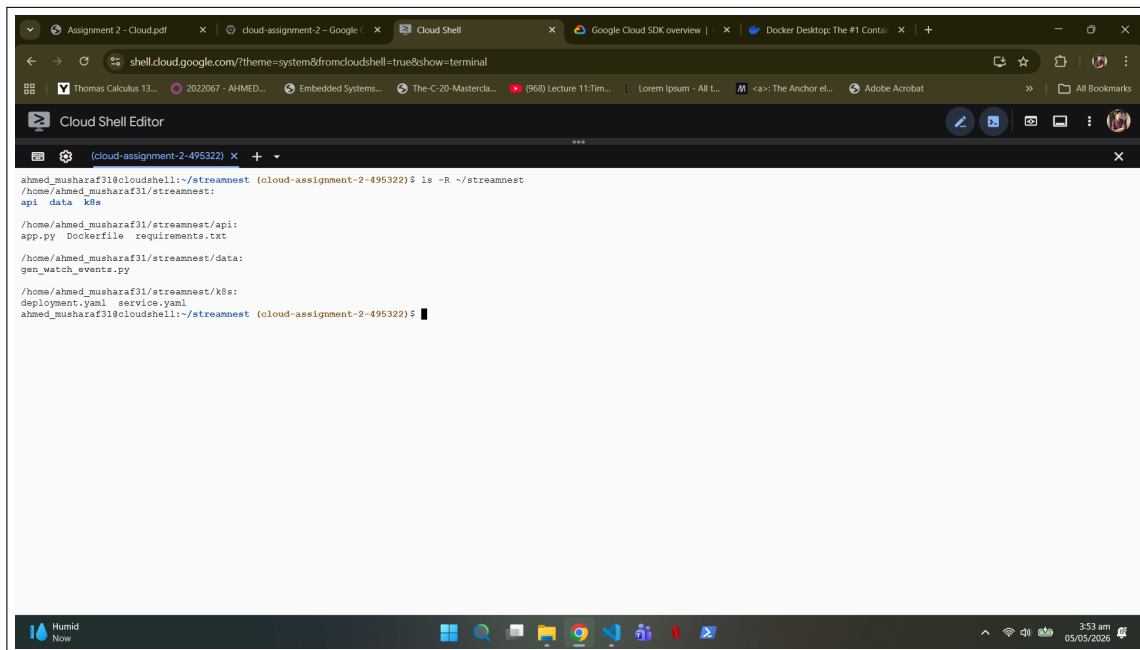


Figure 4: Project directory tree in Cloud Shell showing `api/`, `k8s/`, and `data/` subfolders with all necessary files created.

4.6. Public API Access

The API is publicly accessible at `http://34.124.155.31`. The `/` endpoint returns the hostname of the specific pod that handled the request, making load balancing directly observable by refreshing the browser. Figures 7a and 7b show two consecutive requests being served by different pods.

4.7. Horizontal Scaling

Scaling to 6 replicas required a single command and took under 90 seconds:

```
kubectl scale deployment catalog-api --replicas=6
```

```

Running in 1f5ca15340eb
Removing intermediate container 1f5ca15340eb
bb8048d30b74
Step 8/8 : CMD ["nginx", "-b", "0.0.0.0:8080", "-w", "2", "app:app"]
Running in 762e1659f721
Removing intermediate container 762e1659f721
8e8f1d5db414
Successfully built 8e8f1d5db414
Successfully tagged asia-southeast1-docker.pkg.dev/cloud-assignment-2-495322/streamnest-repo/catalog-api:v1
Push
Pushing asia-southeast1-docker.pkg.dev/cloud-assignment-2-495322/streamnest-repo/catalog-api:v1
The push refers to repository [asia-southeast1-docker.pkg.dev/cloud-assignment-2-495322/streamnest-repo/catalog-api]
4f3559827d01: Preparing
5d58f54be9c: Preparing
8bfc993782b5: Preparing
60cf237be41c: Preparing
2360b28a4660: Preparing
248c30140986: Preparing
falaec823035: Preparing
6d7c150df58d: Preparing
60cf237be41c: Pushed
8bfc993782b5: Pushed
4f3559827d01: Pushed
2360b28a4660: Pushed
5d58f54be9c: Pushed
falaec823035: Pushed
248c30140986: Pushed
6d7c150df58d: Pushed
v1: digest: sha256:3ee800844e6c3e2246943b5bd97b5851a450960b62a9215974b1da0bad3a62b4 size: 1991
DONE
ID: b5a04caa-ec0d-44bc-9937-b872f5142291
CREATE_TIME: 2026-05-04T22:55:45+00:00
DURATION: 45s
SOURCE: gs://cloud-assignment-2-495322_cloudbuild/source/1777935342.463785-0e2d69575af439fbc9984787a99c694.tgz
IMAGE: asia-southeast1-docker.pkg.dev/cloud-assignment-2-495322/streamnest-repo/catalog-api:v1
STATUS: SUCCESS
ahmed_musharaF31@cloudshell:~/streamnest/api (cloud-assignment-2-495322) $

```

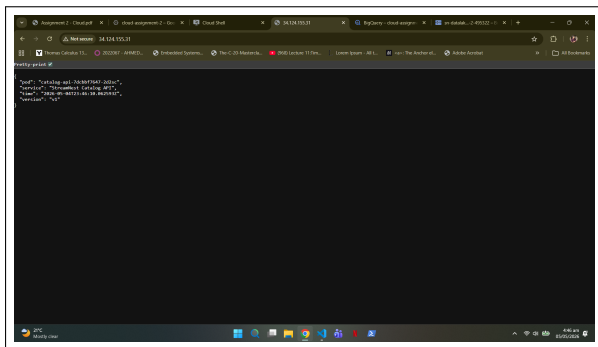
Figure 5: Cloud Build successfully pushing catalog-api:v1 to Artifact Registry, followed by all catalog-api pods confirmed in Running state.

```

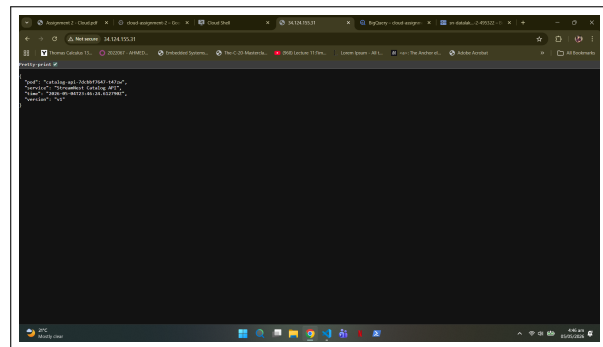
ahmed_musharaF31@cloudshell:~/streamnest/data$ cd ~/streamnest/data
python3 gen_watch_events.py
head -5 watch_events.csv
wc -l watch_events.csv
Wrote 50000 rows to watch_events.csv
event_id,user_id,content_id,watch_duration_sec,device_type,region,event_timestamp
3eebcb6c-660-49e8-ab57-7e2f5d8a1c6,u1258,c001,7145,Browser,Philippines,2026-04-29T00:19:26
cf31ee69-f2c3-4378-8992-124f933dfe4b5,u4141,c007,2769,Android,Pakistan,2026-04-18T15:14:56
67283912-1989-4e9c-94f0-af21e73fe936,u308,c018,5596,SmartTV,Thailand,2026-04-26T04:58:45
9040ac38-b652-4015-a1c8-4e6de045032,u4490,c013,6539,Browser,Indonesia,2026-04-06T12:06:04
50001 watch_events.csv
ahmed_musharaF31@cloudshell:~/streamnest/data$

```

Figure 6: Generating the 50,000-row watch events dataset with gen_watch_events.py and verifying the CSV header and row count in Cloud Shell.



(a) Request served by pod #1.



(b) Same request served by pod #2.

Figure 7: Load balancing in action — consecutive requests to / return different pod hostnames, confirming traffic is distributed across replicas.

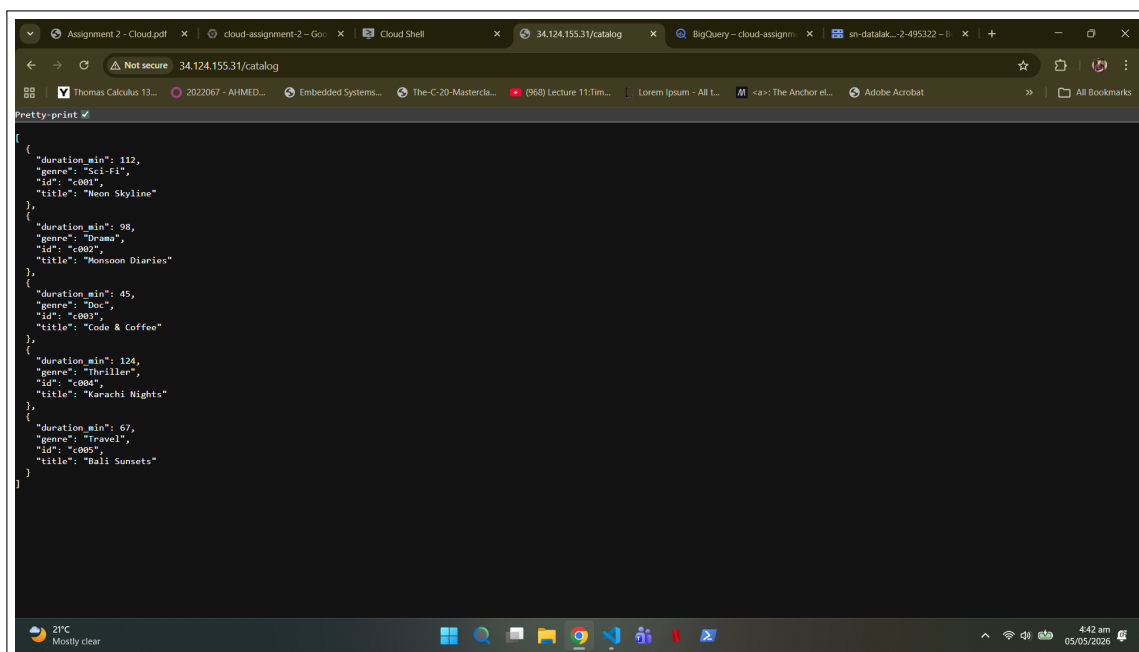


Figure 8: Browser response from GET /catalog — the full JSON content catalog returned by the API.

GKE Autopilot automatically provisioned an additional node to schedule the extra pods — no manual node configuration was required.

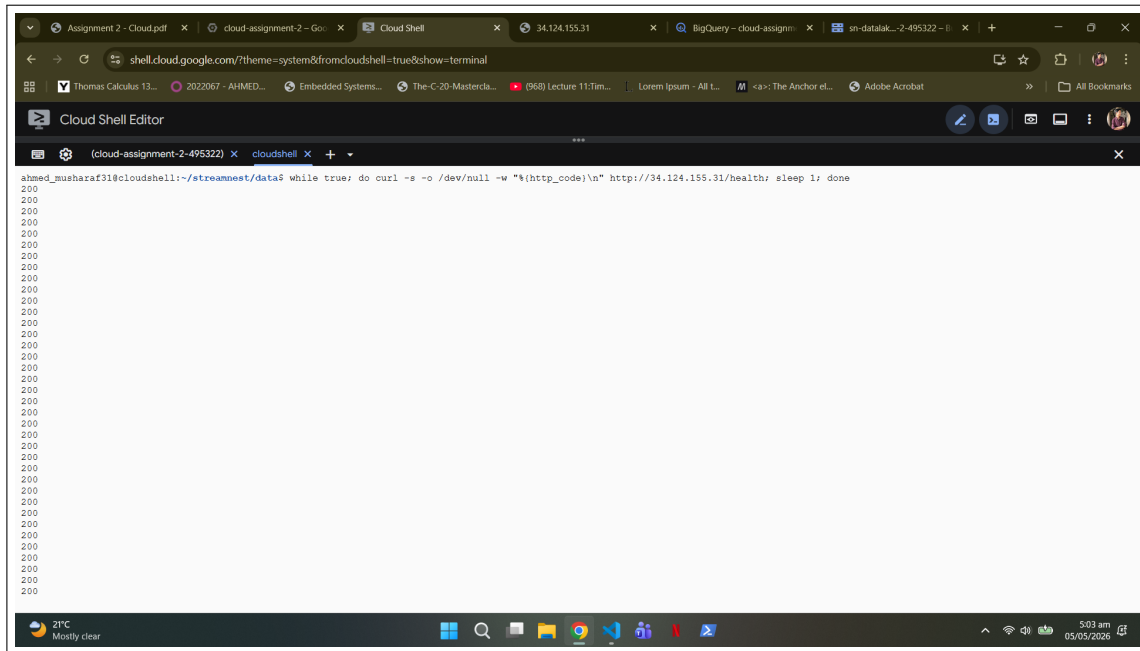


Figure 9: Health-check curl loop running in Cloud Shell confirming all `catalog-api` replicas are responsive during the horizontal scaling test.

4.8. Zero-Downtime Rolling Redeploy

A version 2 image was built (with a visible catalog change) and the deployment was updated:

```
gcloud builds submit --tag \
  asia-southeast1-docker.pkg.dev/$PROJECT_ID/
  streamnest-repo/catalog-api:v2 .

kubectl set image deployment/catalog-api catalog-api=$IMAGE_V2
kubectl rollout status deployment/catalog-api
```

A continuous health-check loop ran in a parallel terminal during the rollout. Figures 10 and 11 show the v2 image build and the unbroken stream of HTTP 200 responses that prove zero downtime.

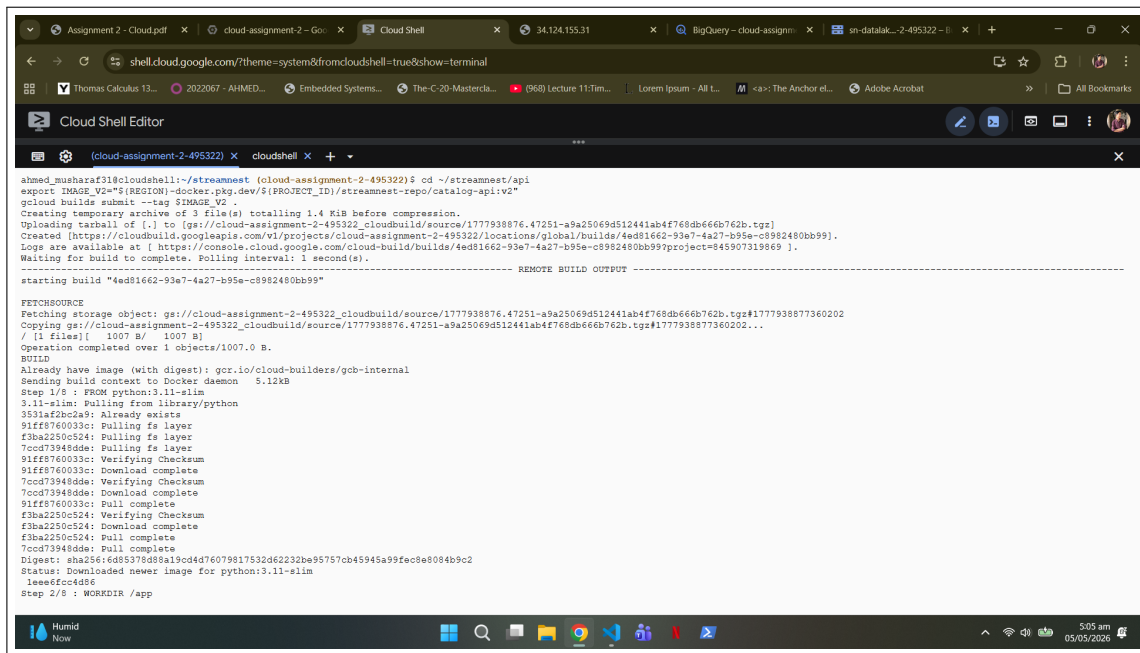


Figure 10: Cloud Build successfully building and pushing `catalog-api:v2` to Artifact Registry — the updated image that triggers the zero-downtime rolling update.

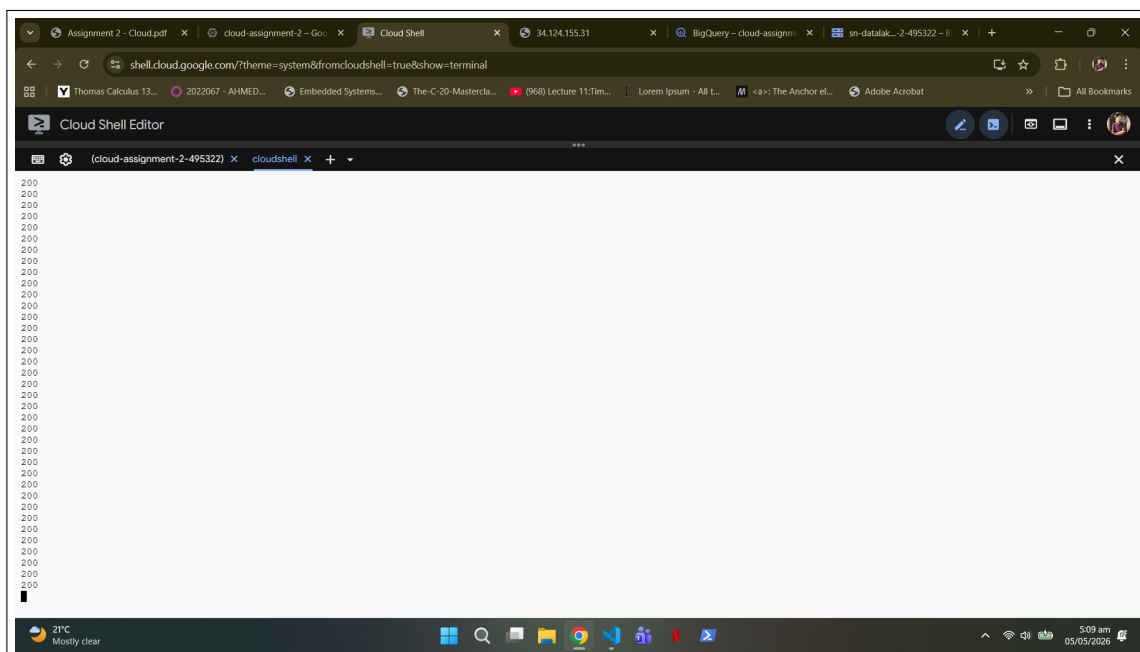


Figure 11: Continuous HTTP 200 responses logged during the rolling redeploy — no failed requests at any point, confirming zero downtime.

5. Data Lakehouse on GCS + BigQuery: Implementation

5.1. Why a Lakehouse Architecture

StreamNest’s data problem requires handling two workload types simultaneously: real-time event ingestion at scale, and ad-hoc analytical queries for the data science team. A pure data warehouse (BigQuery only) requires all data to be loaded and structured upfront — expensive and inflexible for raw event logs arriving in varied formats. A pure data lake (Cloud Storage only) has no query layer, leaving data scientists writing custom parsers.

A **data lakehouse** combines both: raw data is stored cheaply in Cloud Storage (the lake), and BigQuery’s external table feature provides a warehouse-grade SQL layer directly on top without moving any data. This is the optimal architecture for StreamNest’s mixed workload.

5.2. Zone Model

Zone	Technology	Purpose
Raw Zone	Cloud Storage	Immutable, schema-free event logs in original format
Curated Zone	BigQuery native tables	Clean, aggregated, schema-enforced, optimised for query

Table 2: Data lakehouse zone model for StreamNest.

5.3. Step 1: Raw Dataset in Cloud Storage

A realistic simulated dataset of **50,000 watch events** was generated with the following schema — matching StreamNest’s actual event structure:

```
event_id, user_id, content_id, watch_duration_sec,
device_type, region, event_timestamp
```

The dataset covers 5,000 unique users, 20 content titles, 5 device types (**Android**, **iOS**, **WebTV**, **SmartTV**, **Browser**), and 6 Southeast Asian regions over a 30-day window. It was uploaded to a dedicated GCS bucket as the raw ingestion layer:

```
export BUCKET="sn-datalake-${PROJECT_ID}"

gsutil mb -l asia-southeast1 -p $PROJECT_ID gs://$BUCKET

gsutil cp watch_events.csv \
gs://$BUCKET/raw/watch_events/watch_events.csv
```

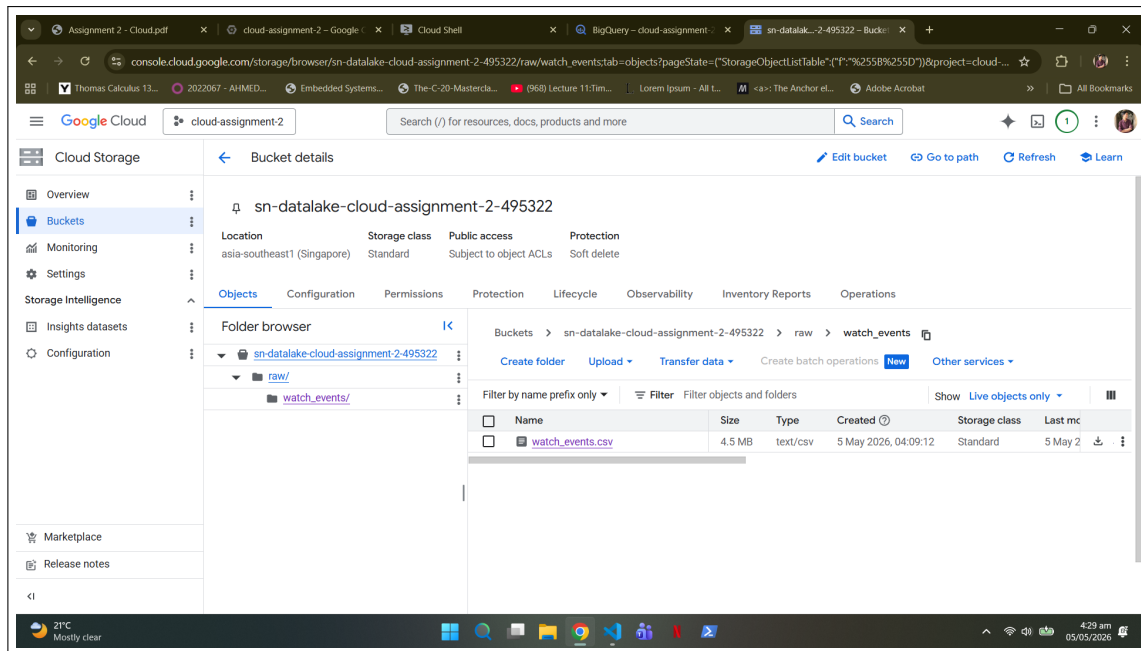


Figure 12: Cloud Storage bucket showing `watch_events.csv` (4.3 MiB, 50,000 rows) stored under the `raw/watch_events/` path — the raw ingestion zone.

5.4. Step 2: BigQuery External Table

An external table was created over the GCS file, enabling SQL queries without data movement. BigQuery reads the CSV directly from Cloud Storage at query time — the raw zone data remains immutable and untouched.

```
CREATE OR REPLACE EXTERNAL TABLE
  `cloud-assignment-2-495322.streamnest_lake.watch_events_ext` (
    event_id          STRING,
    user_id           STRING,
    content_id        STRING,
    watch_duration_sec INT64,
    device_type       STRING,
    region            STRING,
    event_timestamp    TIMESTAMP
  )
OPTIONS (
  format = 'CSV',
  uris    = ['gs://sn-datalake-cloud-assignment-2-495322/
             raw/watch_events/*.csv'],
  skip_leading_rows = 1
);
```

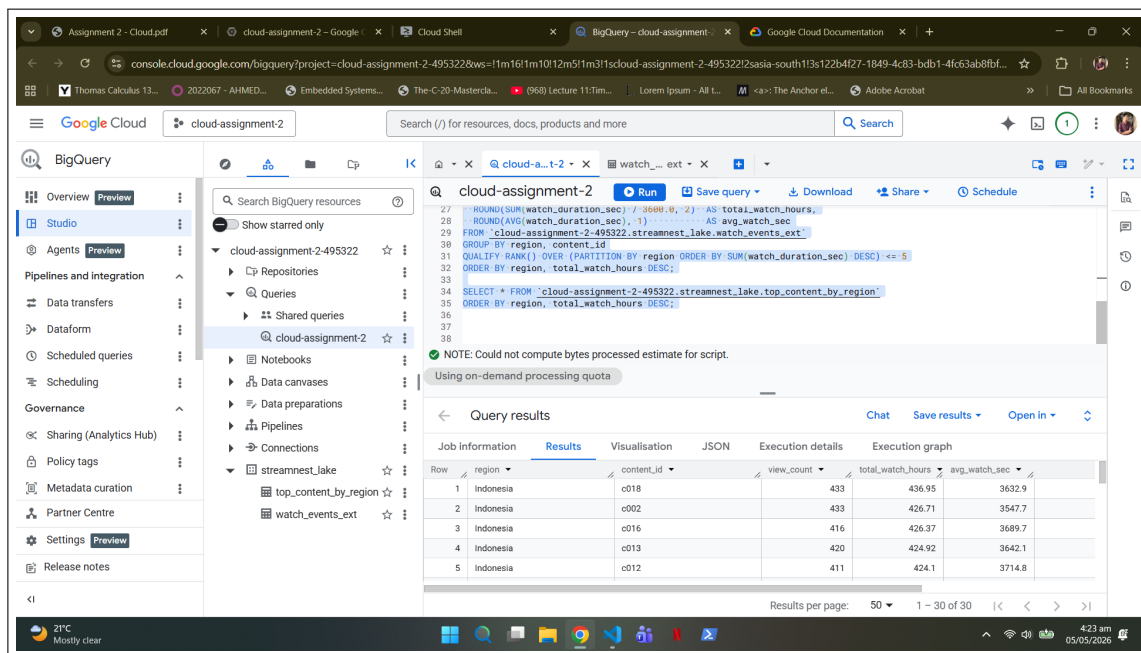



Figure 13: BigQuery query editor showing the aggregation SQL executing against `watch_events_ext`, with the `streamnest_lake` dataset (containing both tables) visible in the left sidebar.

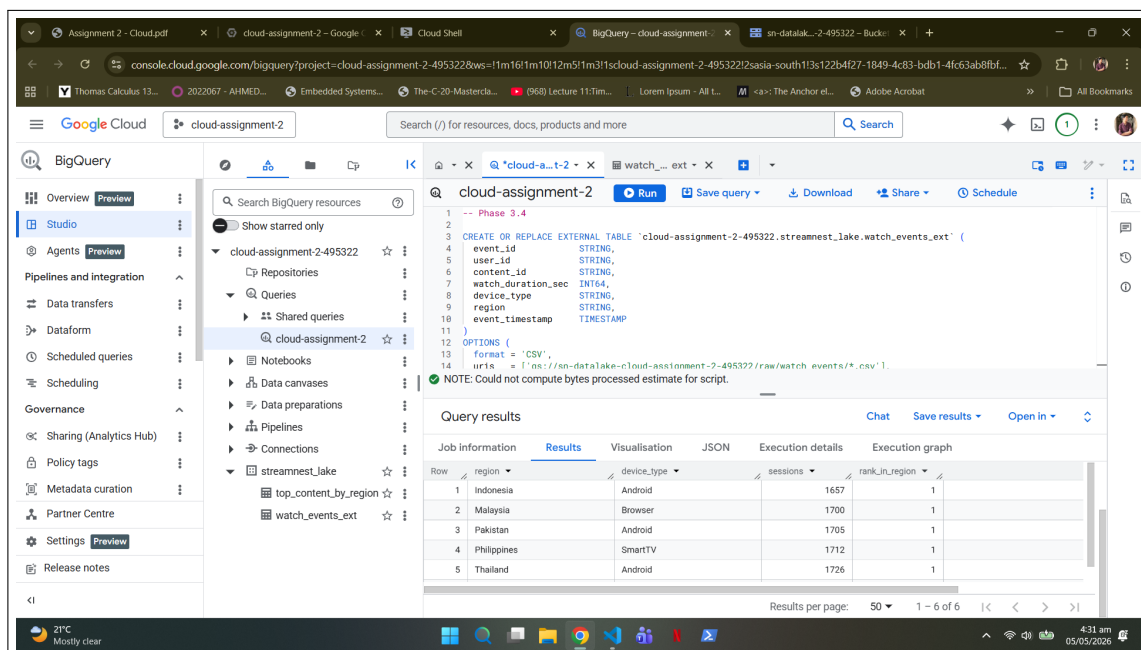


Figure 14: `CREATE EXTERNAL TABLE` statement for `watch_events_ext` with a sample of query results confirming data is accessible directly from Cloud Storage — no ETL or data movement required.

5.5. Step 3: Curated Aggregated Table

A native BigQuery table was created to answer a core business question: **“What are the top 5 most-watched content titles in each region?”** This directly feeds StreamNest’s content recommendation engine and executive reporting. Figures 15 and 16 show the table creation confirmation and the final aggregated data.

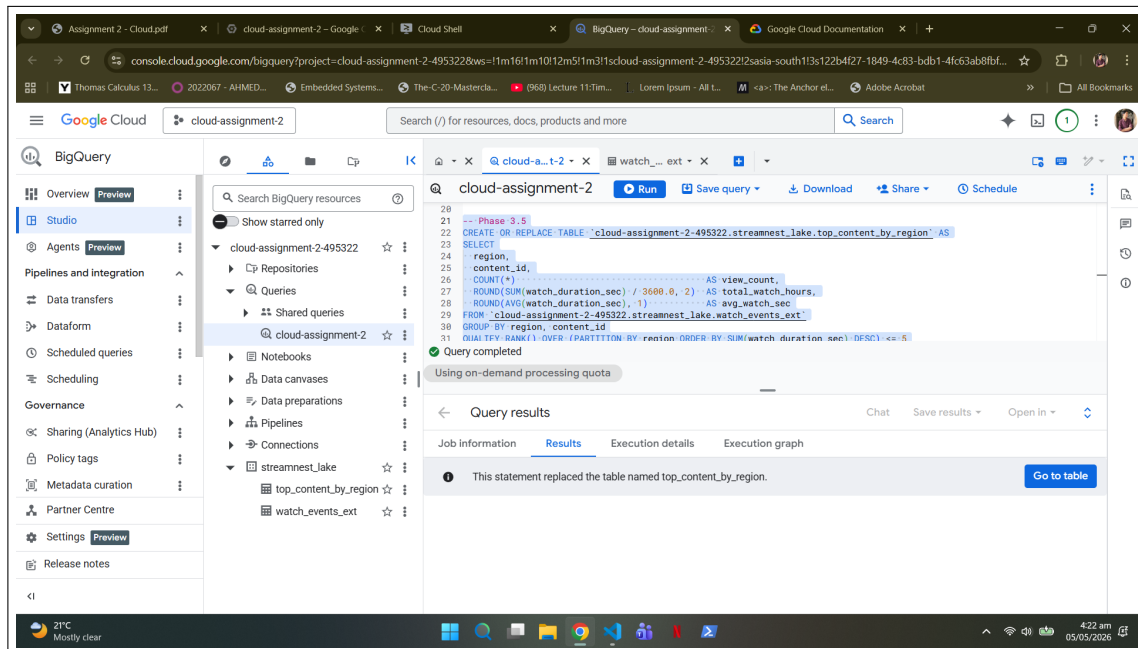


Figure 15: BigQuery confirming successful creation of the `top_content_by_region` curated table — the *Query completed* message confirms the native table was populated from the aggregation.

The SQL used to produce this table is as follows:

```
CREATE OR REPLACE TABLE
  'cloud-assignment-2-495322.streamnest_lake.
    top_content_by_region' AS
SELECT
  region,
  content_id,
  COUNT(*) AS view_count,
  ROUND(SUM(watch_duration_sec)/3600.0, 2) AS total_watch_hours
  ,
  ROUND(AVG(watch_duration_sec), 1) AS avg_watch_sec
FROM `cloud-assignment-2-495322.streamnest_lake.watch_events_ext`
GROUP BY region, content_id
QUALIFY
  RANK() OVER (
    PARTITION BY region
    ORDER BY SUM(watch_duration_sec) DESC
```

```
) <= 5
ORDER BY region, total_watch_hours DESC;
```

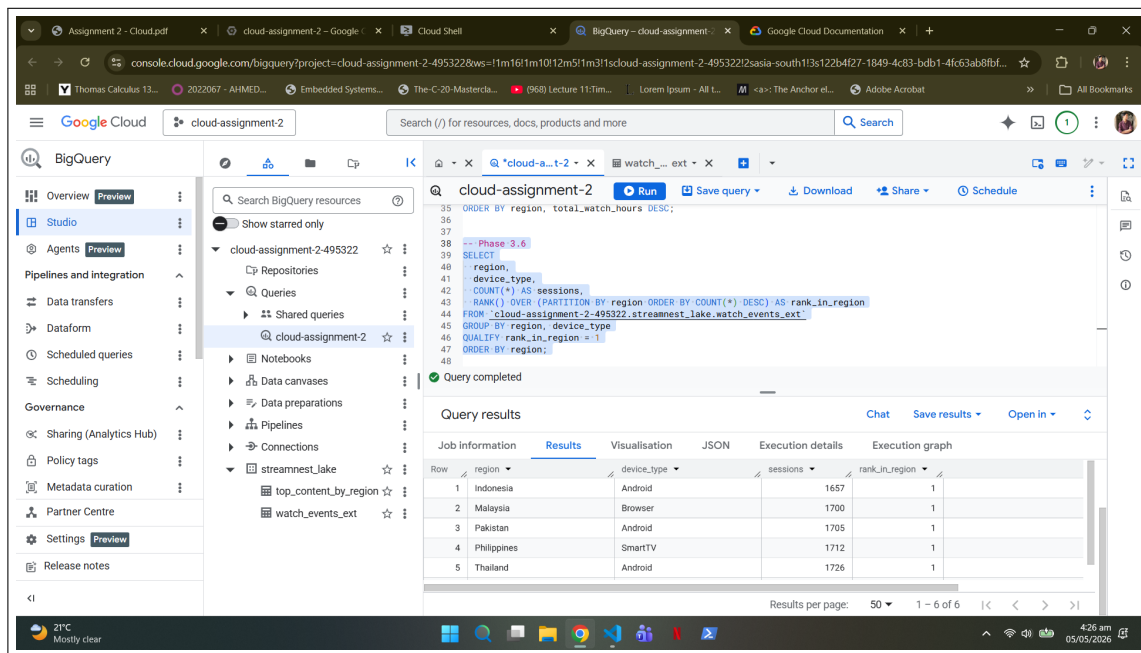


Figure 16: Final `top_content_by_region` table preview showing the top content per region ranked by total watch hours — ready for the recommendation team to consume.

The resulting table gives the data science team a clean, pre-aggregated view they can immediately use for recommendation model training, content acquisition decisions, and regional marketing campaigns.

6. Scalability & Cost Analysis

6.1. GKE Autopilot Scaling

- **Manual scaling:** `kubectl scale` adjusts replicas in under 90 seconds, with Autopilot auto-provisioning nodes as needed — no capacity planning required.
- **Automatic scaling:** A Horizontal Pod Autoscaler (HPA) can be added in production to scale pods on CPU utilisation or custom metrics (e.g., requests-per-second), enabling hands-off scaling during peak streaming hours.
- **Zero-downtime updates:** The `maxUnavailable: 0` rolling update strategy guarantees no user-visible disruption during any deployment.

6.2. BigQuery Scaling

BigQuery is a fully serverless, distributed query engine that scales automatically to petabytes with no infrastructure changes. There is no capacity planning — query slots scale on demand. On-demand pricing means **zero cost when idle**, making it ideal for a data science team with variable query patterns.

6.3. Cloud Storage Cost Optimisation

GCS lifecycle rules reduce storage cost progressively as data ages:

Age	Storage Class	Approx. Cost
0–30 days	Standard	\$0.020/GB/month
30–90 days	Nearline	\$0.010/GB/month
90+ days	Coldline	\$0.004/GB/month

Table 3: GCS lifecycle policy — 80% cost reduction for data older than 90 days.

6.4. Estimated Monthly Cost

Service	Resource	Est. Monthly Cost
GKE Autopilot	3 pods $\times$ 0.25 CPU / 256 Mi	~\$25–40
Cloud Load Balancer	1 rule + data processing	~\$18–20
Artifact Registry	2 image tags stored	~\$0.10
Cloud Storage	4.3 MiB raw data	<\$0.01
BigQuery	On-demand queries on 50K rows	<\$0.01
Cloud Build	2 builds (free tier: 120 min/day)	\$0.00
Total PoC		~\$43–60/month

Table 4: Estimated monthly cost for the proof-of-concept deployment.

At production scale (2M+ users), costs scale **sub-linearly** due to bin-packing efficiency and BigQuery’s shared compute model, meaning cost-per-user decreases as StreamNest grows.

7. Security & Operations

7.1. Identity and Access Management

- **Workload Identity:** GKE pods authenticate to GCP APIs using Workload Identity, binding a Kubernetes service account to a GCP service account — eliminating static API keys inside containers.
- **Least-privilege IAM:** Each microservice gets its own GCP service account with only the permissions it needs. The Catalog API has no BigQuery access; the data pipeline has no GKE access.

7.2. Network Security

- **Private GKE cluster** (production): nodes have no public IPs; access is via Private Google Access and authorised networks only.
- **VPC Service Controls** around BigQuery and GCS prevent data exfiltration even if credentials are compromised.
- **HTTPS termination** at the Load Balancer with Google-managed SSL certificates.

7.3. Data Security

- All data in Cloud Storage and BigQuery is **encrypted at rest by default** using Google-managed AES-256 keys. Customer-Managed Encryption Keys (CMEK) can be applied to sensitive data such as billing records.
- BigQuery **column-level security** can restrict access to PII fields (e.g., `user_id`) to authorised roles only.

7.4. Observability

- **Cloud Logging:** All container stdout/stderr is automatically captured and queryable; BigQuery audit logs track every query.
- **Cloud Monitoring:** GKE dashboards expose CPU, memory, and requests-per-second metrics. Alerting policies can page on-call engineers if pod crash rate or API latency exceeds defined thresholds.

8. Roadmap to Production

The proof-of-concept demonstrates the architecture works end-to-end. Moving to full production requires the following additions in priority order:

1. **Real-time streaming pipeline:** Replace the CSV batch upload with **Pub/Sub** → **Dataflow** → **BigQuery** to ingest watch events in real time, reducing data latency from hours to seconds.
2. **CI/CD pipeline:** Connect GitHub to **Cloud Build** triggers so every merge to `main` automatically tests, builds a new container image, and deploys it to GKE — eliminating all manual deployment steps.
3. **Transformation layer with dbt:** Adopt **dbt** for SQL transformations from raw to curated zones. dbt provides version control, automated testing, and lineage documentation for all data pipelines.
4. **BI dashboards:** Connect **Looker Studio** to the curated BigQuery tables to give executives self-service dashboards with no engineering involvement.

5. **ML-powered recommendations:** Feed curated watch-history tables into **Vertex AI** to train personalised recommendation models, replacing static content browsing with dynamic, user-specific suggestions.
6. **Additional microservices:** Containerise the authentication service and billing service following the same Dockerfile + GKE deployment pattern, and add them to the existing cluster as independent Deployments.

9. Conclusion

This proposal presents a production-ready architecture for StreamNest’s migration to Google Cloud Platform, grounded in a working proof-of-concept that demonstrates each component end-to-end. The two-pillar approach — GKE Autopilot for containerised microservices and a GCS + BigQuery data lakehouse — directly and simultaneously addresses StreamNest’s two core problems: deployment fragility and analytical data access.

The proof-of-concept shows that a containerised service can be deployed, scaled from 3 to 6 replicas, and redeployed with an updated version — all without a single failed health check. It also shows that 50,000 simulated watch events can be stored cheaply in Cloud Storage and queried via BigQuery SQL in seconds to produce real business insights.

This architecture scales to StreamNest’s current 2 million users and well beyond, with costs that decrease per-user as the platform grows. We recommend beginning the production migration with the CI/CD pipeline and real-time streaming components as the highest-impact next steps.

A. Commands & Manifests Reference

A.1. Kubernetes Deployment Manifest

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: catalog-api
spec:
  replicas: 3
  selector:
    matchLabels: { app: catalog-api }
  strategy:
    type: RollingUpdate
    rollingUpdate: { maxSurge: 1, maxUnavailable: 0 }
  template:
    metadata:
      labels: { app: catalog-api }
    spec:
      containers:
        - name: catalog-api
          image: asia-southeast1-docker.pkg.dev/
            cloud-assignment-2-495322/
            streamnest-repo/catalog-api:v2
          ports: [{ containerPort: 8080 }]
          readinessProbe:
            httpGet: { path: /health, port: 8080 }
          resources:
            requests: { cpu: "250m", memory: "256Mi" }
            limits: { cpu: "500m", memory: "512Mi" }
```

A.2. Kubernetes Service Manifest

```
apiVersion: v1
kind: Service
metadata:
  name: catalog-api-svc
spec:
  type: LoadBalancer
  selector: { app: catalog-api }
  ports:
    - port: 80
      targetPort: 8080
```


A.3. BigQuery DDL — External Table

```
CREATE OR REPLACE EXTERNAL TABLE
  'cloud-assignment-2-495322.streamnest_lake.watch_events_ext' (
    event_id          STRING,
    user_id           STRING,
    content_id        STRING,
    watch_duration_sec INT64,
    device_type       STRING,
    region            STRING,
    event_timestamp   TIMESTAMP
  )
OPTIONS (
  format = 'CSV',
  uris    = ['gs://sn-datalake-cloud-assignment-2-495322/
             raw/watch_events/*.csv'],
  skip_leading_rows = 1
);
```

A.4. BigQuery DDL — Curated Aggregated Table

```
CREATE OR REPLACE TABLE
  'cloud-assignment-2-495322.streamnest_lake.
  top_content_by_region'
AS
SELECT
  region,
  content_id,
  COUNT(*) AS view_count,
  ROUND(SUM(watch_duration_sec)/3600.0, 2) AS total_watch_hours,
  ROUND(AVG(watch_duration_sec), 1) AS avg_watch_sec
FROM 'cloud-assignment-2-495322.streamnest_lake.watch_events_ext'
GROUP BY region, content_id
QUALIFY
  RANK() OVER (
    PARTITION BY region
    ORDER BY SUM(watch_duration_sec) DESC
  ) <= 5
ORDER BY region, total_watch_hours DESC;
```

B. Overleaf Upload Checklist

Upload the following to your Overleaf project before compiling:

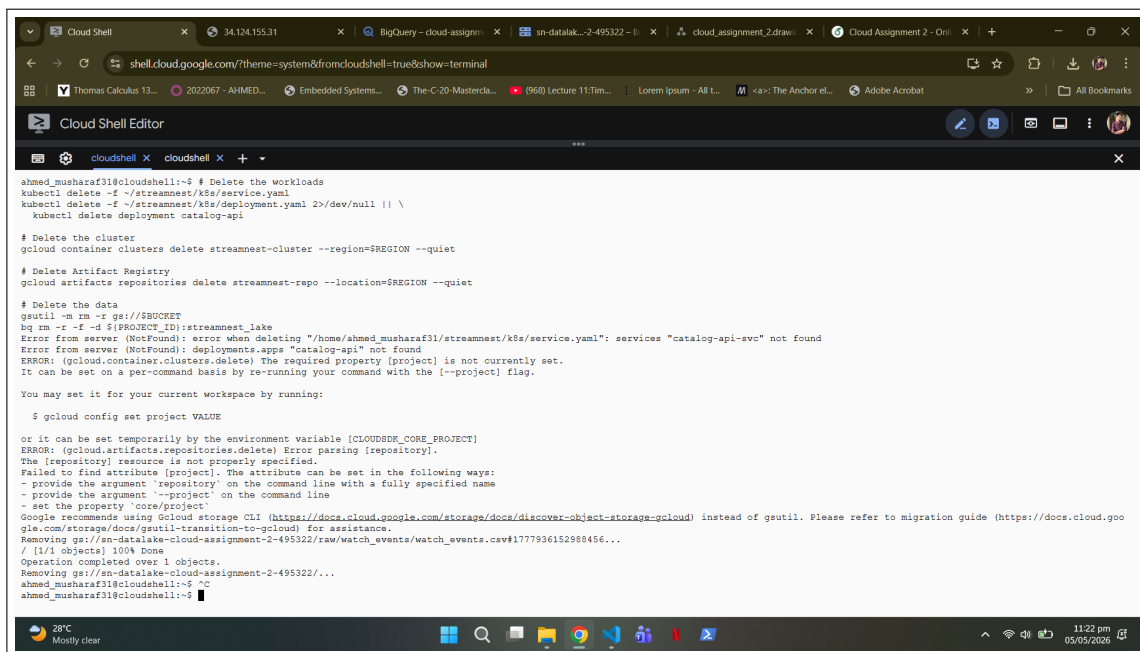
File	Upload location	Used in
architecture.png	Root folder	Section 3
screenshots/1.png	screenshots/ folder	Section 1
screenshots/4.png	screenshots/ folder	Section 4
screenshots/11.png	screenshots/ folder	Section 4
screenshots/13.png	screenshots/ folder	Section 4
screenshots/14.png	screenshots/ folder	Section 4
screenshots/19.png	screenshots/ folder	Section 5
screenshots/20.png	screenshots/ folder	Section 5
screenshots/21.png	screenshots/ folder	Section 5
screenshots/22.png	screenshots/ folder	Section 5
screenshots/23.png	screenshots/ folder	Section 5
screenshots/28.png	screenshots/ folder	Section 4
screenshots/31.png	screenshots/ folder	Section 4
screenshots/32.png	screenshots/ folder	Section 4
screenshots/34.png	screenshots/ folder	Section 4
screenshots/35.png	screenshots/ folder	Section 4
screenshots/40.png	screenshots/ folder	Section 4
screenshots/42.png	screenshots/ folder	Appendix C
screenshots/43.png	screenshots/ folder	Appendix C

Table 5: Files to upload to Overleaf before compiling.

C. Resource Cleanup

All proof-of-concept resources were deleted after the demonstration to prevent ongoing billing charges. The teardown removed every GCP resource provisioned during the assignment in the following order:

1. Kubernetes Service and Deployment (via `kubectl delete`)
2. Cloud Storage bucket (`sn-datalake-cloud-assignment-2-495322`) and all objects
3. Artifact Registry repository (`streamnest-repo`)
4. GKE Autopilot cluster (`streamnest-cluster`)
5. BigQuery dataset (`streamnest_lake`) including all tables



```

ahmed_musharaf31@cloudshell:~$ # Delete the workloads
kubectl delete -f ~/streamnest/k8s/service.yaml
kubectl delete -f ~/streamnest/k8s/deployment.yaml 2>/dev/null || \
  kubectl delete deployment catalog-api

# Delete the cluster
gcloud container clusters delete streamnest-cluster --region=$REGION --quiet

# Delete Artifact Registry
gcloud artifacts repositories delete streamnest-repo --location=$REGION --quiet

# Delete the data
gsutil -m rm -r gs://$BUCKET
bq rm -r -f -d $(PROJECT_ID):streamnest_lake
Error from server (NotFound): error when deleting "/home/ahmed_musharaf31/streamnest/k8s/service.yaml": services "catalog-api-svc" not found
Error from server (NotFound): deployment.apps "catalog-api" not found
ERROR: (gcloud.container.clusters.delete) The required property [project] is not currently set.
It can be set on a per-command basis by re-running your command with the [--project] flag.

You may set it for your current workspace by running:

$ gcloud config set project VALUE

or it can be set temporarily by the environment variable [CLOUDSDK_CORE_PROJECT]
ERROR: (gcloud.artifacts.repositories.delete) Error parsing [repository].
The [repository] resource is not properly specified.
Failed to find attribute [project]. The attribute can be set in the following ways:
- provide the argument 'repository' on the command line with a fully specified name
- provide the argument '--project' on the command line
- set the property 'core/project'
Google recommends using Gcloud storage CLI (https://docs.cloud.google.com/storage/docs/discover-object-storage-gcloud) instead of gsutil. Please refer to migration guide (https://docs.cloud.google.com/storage/docs/gsutil-transition-to-gcloud) for assistance.
Removing gs://sn-datalake-cloud-assignment-2-495322/raw/watch_events/watch_events.csv#177936152988456...
// [1/1 objects] 100% Done
Operation completed over 1 objects.
Removing gs://sn-datalake-cloud-assignment-2-495322/...
ahmed_musharaf31@cloudshell:~$ ^C
ahmed_musharaf31@cloudshell:~$

```

Figure 17: Cloud Shell showing deletion of the GKE workloads (service and deployment) and the Cloud Storage bucket during proof-of-concept teardown.

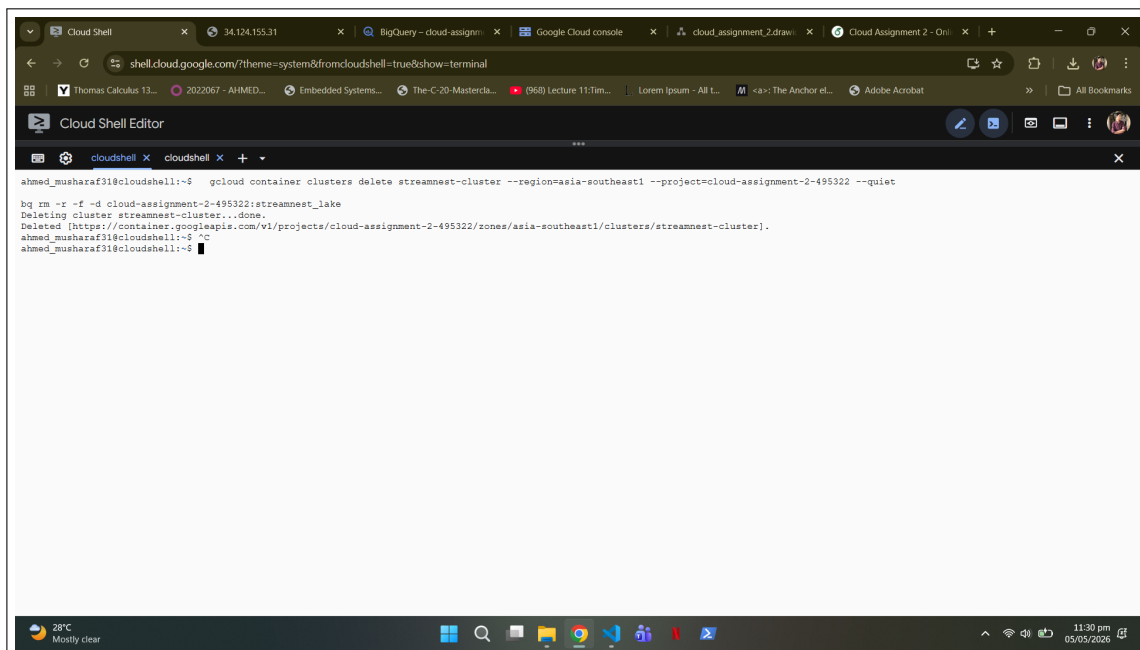


Figure 18: Successful deletion of the GKE Autopilot cluster (**streamnest-cluster**) and BigQuery dataset — all proof-of-concept resources removed to prevent ongoing billing.